Reviewer Pack — Minimal Monodromy Rank of Algebraic Curves Bounds Resolution…

Entry c94dcc3ae3ad · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 14:39:10 UTC

Minimal Monodromy Rank of Algebraic Curves Bounds Resolution Proof Length

Entry ID: `c94dcc3ae3ad` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 14:39:10 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Monodromy Theory) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['For a given CNF formula F, let R(F) be the minimal monodromy rank of its associated algebraic curve over an appropriate field extension. Then, for all instances F with n variables, the resolution proof length t*(F) satisfies the inequality: t*(F) ≤ O(n^2 * R(F)).', 'Equivalently, there exists a polynomial-time computable function f(n) such that for every CNF formula F, R(F) ≤ f(n).', "The function f(n) is a constructive mapping from instances of size n to the associated algebraic curve's monodromy rank."]

Rationale (proposer's reasoning):

> ['Monodromy theory provides an algebraic invariant that captures the structure of transcendental functions and their extensions. The conjecture suggests that this invariant might be useful for understanding the complexity of resolution proofs, as it has been known to capture complexity in other settings.', 'Algebraic curves with higher monodromy rank may require more complex manipulations during proof construction, leading to longer resolution proof lengths.', 'This bridge could potentially expose a new method for proving lower bounds on resolution proof length.']

Taxonomy category: `Geometric Complexity Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3cf2c9afb2ebdec5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across 30 seeds, the mean resolution proof length t*(F) of CNF formulas F with n variables is less than or equal to O(n^2* R(F)) for all F, where R(F) is the minimal monodromy rank. The conjecture is falsified if any seed produces a mean metric greater than O(n^2 * R(F)).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"minimal monodromy rank" AND "resolution proof length"` - `"algebraic geometry" AND "resolution proof complexity"` - `"polynomial-time computable function" AND "monodromy theory"`

Top relevant hits considered: - [s2:10.4230/LIPIcs.STACS.2025.8] Tropical Proof Systems: Between R(CP) and Resolution - [s2:eaa2c3b560b1091e91624f9005fbeb309636679b] Foundations of algebraic geometry and resolution of singularities

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=1.9s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([f'x{i}', f'~x{i}']) for i in range(n)]
            if len(set(clause)) == 1:
                continue
            random.shuffle(clause)
            clauses.append('(' + ' & '.join(clause) + ')')
        return ' | '.join(clauses)

    def resolution_length(cnf):
        stack = []
        while cnf:
            literals = set()
            for clause in cnf.split(' | '):
                if '(' in clause and ')' in clause:
                    continue
                literals.update(clause.split(' & '))
            literal = random.choice(list(literals))
            new_clauses = []
```

```
            for clause in cnf.split(' | '):
                if literal not in clause and f'~{literal}' not in clause:
                    new_clauses.append(clause)
                elif literal in clause:
                    continue
                else:
                    other_clause = clause.replace(f'~{literal}', '')
                    other_literals = set(other_clause.split(' & '))
                    for l in literals:
                        if l != literal and f'~{l}' not in other_literals:
                            new_clauses.append(f'({other_clause} & {l})')
            cnf = ' | '.join(new_clauses)
            stack.append(literal)
        return len(stack)

    def monodromy_rank(n):
        # Placeholder for actual mapping
        return n

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        cnf = generate_cnf(n)
        t_F = resolution_length(cnf)
        R_F = monodromy_rank(n)
        if R_F == 0:
            continue
        results.append((t_F, R_F))

    if not results:
        return {
            "metric_name": "resolution_proof_length",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    mean_t_F = sum(t_F for t_F, _ in results) / len(results)
    max_R_F = max(R_F for _, R_F in results)
    conjecture_holds = all(t_F <= n**2 * R_F for t_F, R_F in results)

    return {
        "metric_name": "resolution_proof_length",
        "metric_value": mean_t_F,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"n={max_R_F}, t*(F)={mean_t_F}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
```

```
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(r["conjecture_holds"] for r in results):
        mean_t_F = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        result = f"RESULT: SUPPORTED mean={mean_t_F} std=0 support_fraction={support_fraction}"
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        counterexample = r["counterexample"]
        result = f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first

    print(result)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33672c96.py", line 96, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33672c96.py", line 63, in run_trial
    t_F = resolution_length(cnf)
          ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_33672c96.py", line 39, in resolution_le
    literal = random.choice(list(literals))
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 347, in choice
    raise IndexError('Cannot choose from an empty sequence')
IndexError: Cannot choose from an empty sequence
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing any data, which means we cannot determine if the conjecture is supported or falsified based on the pre-registered support conditions. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12519
2	preregistration	ollama_remote	glm4:latest	0	0	6055
3	novelty	ollama_remote	glm4:latest	0	0	4748
4	novelty	ollama_remote	glm4:latest	0	0	6289
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15799
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10367
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13225
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30012
9	judge	ollama_remote	glm4:latest	0	0	53258

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 152272 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c94dcc3ae3ad.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c94dcc3ae3ad.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c94dcc3ae3ad.tar.gz` (if generated)